



YOHPAL LIVE RELEASE 1.0B — Money, Security & Wallet Reconciliation Patch.

Daniel Macharia <danielmacharia43@gmail.com>
To: <daniel@ics.ac.ke>

Thu, 9 Jul at 22:47

Below is **YOHPAL LIVE RELEASE 1.0B — Money, Security & Wallet Reconciliation Patch.**

It targets the audit's Critical money/fraud blockers: client-controlled gift amounts, spoofable rewarded-ad rewards, weak Firestore writes for gifts/videoStats/referralEvents, and missing wallet reconciliation UX.

YohPal Live — Production Readiness Review.pdf

```
// functions/src/gifts/validatedGiftPurchase.ts
import * as functions from "firebase-functions/v2/https";
import * as admin from "firebase-admin";

export const validatedGiftPurchase = functions.onCall(async (request) => {
  if (!request.auth) {
    throw new functions.HttpsError("unauthenticated", "Login required.");
  }

  const senderId = request.auth.uid;
  const { liveSessionId, creatorId, giftId } = request.data;

  const giftCatalog: Record<string, number> = {
    rose: 10,
    star: 50,
    crown: 100,
  };

  const amount = giftCatalog[giftId];

  if (!liveSessionId || !creatorId || !amount) {
    throw new functions.HttpsError("invalid-argument", "Invalid gift.");
  }

  await admin.firestore().runTransaction(async (tx) => {
    const walletRef = admin.firestore().collection("wallets").doc(senderId);
    const creatorWalletRef = admin.firestore().collection("wallets").doc(creatorId);
    const sessionRef = admin.firestore().collection("liveSessions").doc(liveSessionId);
    const giftRef = sessionRef.collection("gifts").doc();

    const walletSnap = await tx.get(walletRef);
    const balance = walletSnap.data()?.balance ?? 0;

    if (balance < amount) {
      throw new functions.HttpsError("failed-precondition", "Insufficient wallet balance.");
    }

    tx.update(walletRef, {
      balance: admin.firestore.FieldValue.increment(-amount),
      updatedAt: admin.firestore.FieldValue.serverTimestamp(),
    });

    tx.set(giftRef, {
      senderUserId: senderId,
      creatorId,
      giftId,
      amount,
      verified: true,
      createdAt: admin.firestore.FieldValue.serverTimestamp(),
    });

    tx.set(creatorWalletRef, {
      balance: admin.firestore.FieldValue.increment(amount),
      updatedAt: admin.firestore.FieldValue.serverTimestamp(),
    }, { merge: true });

    tx.update(sessionRef, {
      giftTotal: admin.firestore.FieldValue.increment(amount),
      updatedAt: admin.firestore.FieldValue.serverTimestamp(),
    });
  });

  return { status: "success", amount };
});

// functions/src/ads/verifyRewardedAd.ts
```

```

import * as functions from "firebase-functions/v2/https";
import * as admin from "firebase-admin";

export const verifyRewardedAd = functions.onCall(async (request) => {
  if (!request.auth) {
    throw new functions.HttpsError("unauthenticated", "Login required.");
  }

  const uid = request.auth.uid;
  const { adNetwork, adUnitId, verificationToken } = request.data;

  if (!adNetwork || !adUnitId || !verificationToken) {
    throw new functions.HttpsError("invalid-argument", "Missing ad verification payload.");
  }

  // TODO: Replace this with real AdMob / ad-network server-side verification.
  const verified = verificationToken.length > 20;

  if (!verified) {
    throw new functions.HttpsError("permission-denied", "Ad reward not verified.");
  }

  await admin.firestore().collection("rewardedAdEvents").add({
    uid,
    adNetwork,
    adUnitId,
    verified: true,
    rewardAmount: 1,
    createdAt: admin.firestore.FieldValue.serverTimestamp(),
  });

  return { status: "verified", rewardAmount: 1 };
});

// firestore/rules/firestore.rules patch concept

match /liveSessions/{sessionId}/gifts/{giftId} {
  allow read: if request.auth != null;
  allow create, update, delete: if false;
}

match /videoStats/{videoId} {
  allow read: if true;
  allow create, update, delete: if false;
}

match /referralEvents/{eventId} {
  allow read: if request.auth != null;
  allow create: if request.auth != null
    && request.resource.data.uid == request.auth.uid
    && request.resource.data.keys().hasOnly([
      "uid",
      "referralCode",
      "source",
      "createdAt"
    ]);
  allow update, delete: if false;
}

match /liveSessions/{sessionId} {
  allow read: if true;
  allow update: if request.auth != null
    && request.auth.uid == resource.data.creatorId;
}

// lib/features/wallet/wallet_status_screen.dart
import 'package:flutter/material.dart';

class WalletStatusScreen extends StatelessWidget {
  static const routeName = '/wallet-status';

  const WalletStatusScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final uri = Uri.base;
    final status = uri.queryParameters['status'] ?? 'pending';
    final reference = uri.queryParameters['reference'] ?? 'unknown';

    final success = status == 'success';

    return Scaffold(
      appBar: AppBar(title: const Text('Wallet Status')),
      body: Center(
        child: Padding(
          padding: const EdgeInsets.all(24),
          child: Column(

```

```

        mainAxisSize: MainAxisSize.min,
        children: [
          Icon(
            success ? Icons.check_circle : Icons.info,
            size: 64,
          ),
          const SizedBox(height: 16),
          Text(
            success ? 'Payment received' : 'Payment status pending',
            style: Theme.of(context).textTheme.headlineSmall,
          ),
          const SizedBox(height: 8),
          Text('Reference: $reference'),
          const SizedBox(height: 24),
          FilledButton(
            onPressed: () => Navigator.pop(context),
            child: const Text('Return to YohPal'),
          ),
        ],
      ),
    ),
  ),
);
}

// lib/features/wallet/yohpal_web_handoff.dart
import 'package:url_launcher/url_launcher.dart';

class YohPalWebHandoff {
  Future<bool> openWalletCheckout({
    required String checkoutUrl,
    required String returnUrl,
  }) async {
    final uri = Uri.parse('$checkoutUrl?returnUrl=${Uri.encodeComponent(returnUrl)}');

    return launchUrl(
      uri,
      mode: LaunchMode.externalApplication,
    );
  }
}

// lib/features/gifts/gift_service.dart
import 'package:cloud_functions/cloud_functions.dart';

class GiftService {
  final FirebaseFunctions functions;

  GiftService({FirebaseFunctions? functions})
    : functions = functions ?? FirebaseFunctions.instance;

  Future<void> sendGift({
    required String liveSessionId,
    required String creatorId,
    required String giftId,
  }) async {
    await functions.httpsCallable('validatedGiftPurchase').call({
      'liveSessionId': liveSessionId,
      'creatorId': creatorId,
      'giftId': giftId,
    });
  }
}

# Required validation
flutter analyze
flutter test

cd functions
npm run build
npm test

firebase deploy --only functions:validatedGiftPurchase
firebase deploy --only functions:verifyRewardedAd

```

Developer Evidence Required

1. Git branch
2. Commit hash
3. Firestore rules patch proof
4. validatedGiftPurchase deploy proof
5. verifyRewardedAd deploy proof
6. WalletStatusScreen screenshot
7. Gift flow test proof
8. Rewarded-ad verification proof

9. flutter analyze output
10. flutter test output
11. npm run build output
12. npm test output

Next Smart Move

Execute **YOHPAL LIVE RELEASE 1.0C — Feed, Live Discovery & UX Recovery Patch** to wire Search/Live Discovery, add feed retry, pagination, mute, double-tap-like, accessibility semantics, and LiveKit reconnect handling.